Project Report: Flask Calculator API

Group Member

Aaditya Parihar (23070123001) Siddhant Bedre(23070123037)

Akshit Bansal (23070123013) Hardik Dhalla (23070123058)

Summary of the Project

The "Flask Calculator API" is a lightweight, server-side application built using the Python Flask framework. Its primary purpose is to provide a simple RESTful web service for performing basic mathematical operations. The API accepts requests formatted in JSON and returns results in JSON, making it an ideal project for learning and demonstrating the fundamentals of REST API development, endpoint creation, and basic error handling within Flask.

Core Features & Functionality

The API is designed to handle five distinct mathematical operations, each mapped to a specific endpoint:

- **Addition:** Adds two provided numbers.
- **Subtraction:** Subtracts the second number from the first.
- **Multiplication:** Multiplies two provided numbers.
- **Division:** Divides the first number by the second, with an integrated check to prevent division by zero.
- **Square:** Calculates the square of a single provided number.

The service also includes a root (/) endpoint that serves as a welcome message and lists available endpoints for discovery.

Technical Specifications

The project relies on a minimal and focused technology stack:

- **Core Language:** Python (3.x)
- **Web Framework:** Flask
- **Data Format:** JSON (for all request payloads and responses)

API Endpoint Definitions

All calculation endpoints are accessed using the POST method, requiring a JSON body. The API is hosted at the base URL: http://127.0.0.1:5000/.

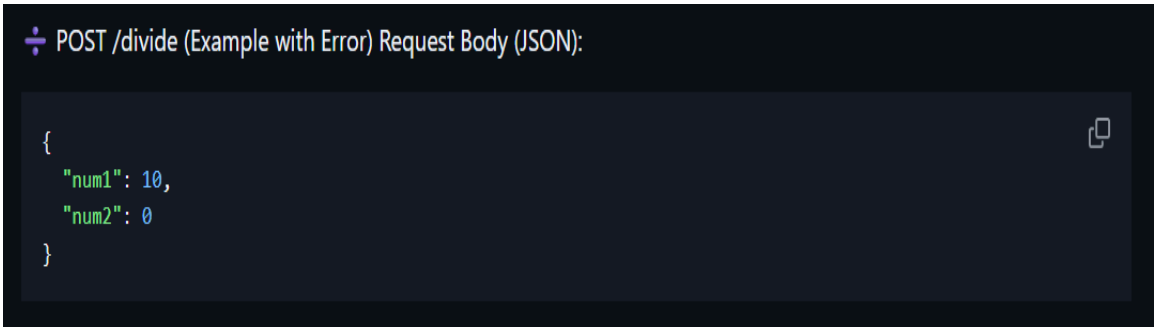
Method	Endpoint	Description	Required JSON Fields
GET	/	Welcome route showing available endpoints	None
POST	/add	Adds two numbers	num1, num2
POST	/subtract	Subtracts num2 from num1	num1, num2
POST	/multiply	Multiplies two numbers	num1, num2
POST	/divide	Divides num1 by num2	num1, num2
POST	/square	Finds the square of a number	num

Example Request & Response :

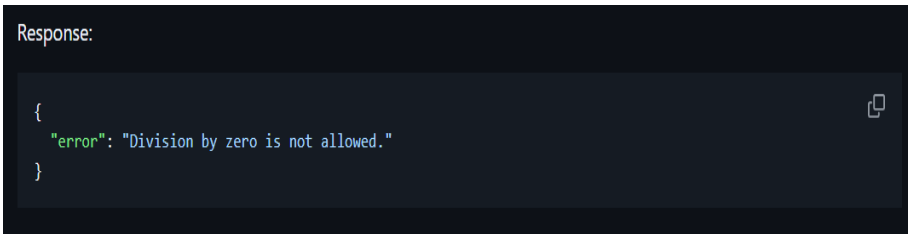




POST /divide (Example with Error) Request Body (JSON):



Response:



Error Handling

The API includes robust error handling to manage invalid requests gracefully. It returns a descriptive JSON error message for the following conditions:

- **Division by Zero:** Triggered if num2 is 0 in a /divide request.
    - *Response:* {"error": "Division by zero is not allowed."}
- **Missing or Invalid Fields:** Triggered if the JSON body is missing num1, num2, or num as required by the endpoint.
- **Non-Numeric Input:** Triggered if the values provided for the numbers are not valid numeric types.
- **Invalid JSON Format:** Triggered if the request body is not well-formed JSON.
    - *Example Response:* {"error": "Invalid input. Please provide valid JSON with 'num1' and 'num2'."}

Setup and Testing

Setup Instructions :-

1. **Clone Repository:** The project is initiated by cloning its Git repository.

```
Bash

git clone https://github.com/your-username/flask-math-api.git
cd flask-math-api
```

2. **Install Dependencies:** The only required dependency is Flask.

```bash
pip install flask
```

3. **Run Application:** The server is started by running the main Python script.

```bash
python app.py
```

Testing Procedures :-

The API can be tested using any API client, such as Postman. The recommended procedure is:

1. Create a new **POST** request.
2. Set the request URL to the desired endpoint (e.g., http://127.0.0.1:5000/multiply).
3. Navigate to the **Body** tab, select **raw**, and choose **JSON** from the dropdown menu.
4. Input the required JSON payload (e.g., {"num1": 10, "num2": 5}).
5. Send the request to receive the JSON response.

Conclusion

This project successfully fulfils its objective of creating a simple, functional, and educational REST API. It serves as a well-documented and fundamental example of API development in Flask, correctly implementing routing, JSON data handling, and essential error checks. It is an excellent starting point for developers new to Flask or back-end web services.